00 — Análise do projeto

Análise feita **antes** de qualquer alteração no código fornecido.

1. Estrutura encontrada

```
SistemaBancario/                      (pasta externa)
├── Discussoes_Refatoracao_e_Testes_texto.pdf   (relatório da atividade anterior)
└── SistemaBancario/                  (projeto Eclipse)
    ├── .project / .classpath / .settings/
    └── src/
        ├── negocio/
        │   ├── Cliente.java
        │   ├── ContaCorrente.java
        │   ├── EntidadeBancaria.java          (interface: getId(), isAtivo())
        │   ├── Repositorio.java               (interface genérica de CRUD)
        │   ├── RepositorioBase.java           (implementação do CRUD em lista)
        │   ├── RepositorioContas.java         (interface + transfereValor)
        │   ├── GerenciadoraClientes.java      (repositório de Cliente)
        │   ├── GerenciadoraContas.java        (repositório de ContaCorrente +
transferência)
        │   ├── SistemaBancario.java           (fachada das operações)
        │   ├── CargaInicialFactory.java       (dados de exemplo para o Main)
        │   ├── IdadeNaoPermitidaException.java
        │   └── Main.java                      (menu de console)
        └── teste/
            └── TesteUnitarioManual.java       (testes sem framework, via main)
```

2. Tecnologias utilizadas

Item	Encontrado
Linguagem	Java. O Eclipse está configurado com *compliance* **1.8** (`.settings/org.eclipse.jdt.core.prefs`). O JDK disponível na máquina é o 17.
Build	**Nenhum** (sem Maven/Gradle). Projeto Eclipse puro.
Framework de testes	O `.classpath` referencia `junit-4.12.jar` e `hamcrest-core-1.3.jar` (caminho `D:/eclipse-workspace/`), mas **nenhuma classe usa JUnit**.
Testes existentes	`teste.TesteUnitarioManual`: 25 verificações manuais com `System.out` (`[OK]` / `[FALHOU]`).
BDD	Nenhum.

Execução dos testes existentes no código **original** (evidência: `evidencias/00-baseline-teste-manual-original.txt`):

```
Total de testes: 25
Falharam: 0
Passaram: 25
```

3. Classes principais e responsabilidades

Classe	Responsabilidade encontrada
`Cliente`	id, nome, idade, email, `boolean ativo`, id da conta. **Valida a idade no construtor**.
`ContaCorrente`	id, saldo, `boolean ativo`. Apenas getters/setters.
`RepositorioBase<T>`	`pesquisa`, `adiciona`, `remove`, `estaAtivo`, `listarTodos`, `limpa` sobre uma `List`.
`GerenciadoraClientes`	Repositório de clientes (herda tudo de `RepositorioBase`).
`GerenciadoraContas`	Repositório de contas + `transfereValor(idOrigem, valor, idDestino)`.
`SistemaBancario`	Fachada: `consultarCliente`, `consultarConta`, `ativarCliente`, `desativarCliente`.
`Main`	Menu de console com as 4 operações acima.

4. Como os requisitos funcionavam no código original

#	Requisito	Situação no código original
1	Consultar cliente	`SistemaBancario.consultarCliente(id)` → devolve o `Cliente` ou `null`.
2	Cadastrar cliente	Só existia `Repositorio.adiciona()`, **sem** verificar id duplicado. `SistemaBancario` não oferecia cadastro.
3	Validar idade	Construtor de `Cliente` lança `IdadeNaoPermitidaException` fora de **[18, 65]**. Porém `setIdade()` **não validava**.
4	Ativar cliente	`SistemaBancario.ativarCliente(id)` → `setAtivo(true)`; `false` se o cliente não existe.
5	Desativar cliente	`SistemaBancario.desativarCliente(id)` → `setAtivo(false)`; `false` se não existe.
6	Situação do cliente	`boolean ativo` / `isAtivo()`; `Repositorio.estaAtivo(id)` devolve `false` tanto para inativo quanto para **inexistente**.
7	Remover cliente	`Repositorio.remove(id)` → `true`/`false`. Não exposto no `SistemaBancario`. Sem restrições.
8	Consultar conta	`SistemaBancario.consultarConta(id)` → `ContaCorrente` ou `null`.
9	Cadastrar conta	Mesmo caso do requisito 2: só `adiciona()`, sem verificar duplicidade.
10	Situação da conta	`boolean ativo` / `isAtivo()`, mesmo problema do requisito 6.
11	Remover conta	`Repositorio.remove(id)`. Não exposto no `SistemaBancario`. Sem restrições.
12	Transferência	`GerenciadoraContas.transfereValor`: exige que as duas contas existam e que `saldo >= valor`. Débito/crédito feitos no repositório via `setSaldo`.

5. Problemas identificados

1. **Sem build nem framework de testes executável**: o JUnit está declarado no Eclipse com caminho absoluto de Windows e não é usado.
2. **Cadastro sem regra de unicidade**: é possível adicionar dois clientes (ou duas contas) com o mesmo id; a consulta passa a devolver sempre o primeiro.
3. **`Cliente.setIdade()` burla a regra de idade**: um cliente válido pode passar a ter 10 anos, contradizendo o comentário da própria classe ("um Cliente nunca existe em estado inválido").
4. **Transferência aceita valor negativo e zero**: `transfereValor(1, -50, 2)` passa na verificação `saldo < valor` e movimenta o dinheiro **no sentido contrário** (credita a origem e debita o destino).
5. **Regra de transferência fora da entidade**: o repositório manipula o saldo via `setSaldo(getSaldo() - valor)`; a conta não "sabe" transferir.
6. **Situação como `boolean`**: `ativo = true/false` não expressa o conceito do domínio e `estaAtivo(id)` confunde "inexistente" com "inativo".
7. **Operações não expostas**: `SistemaBancario` não oferecia cadastro, remoção, transferência nem verificação de situação.

> **Complemento pós-auditoria** (`06-correcoes-pos-auditoria.pdf`): também não foram percebidos nesta análise inicial: `setId` público (permitia quebrar a unicidade), `listarTodos()` expondo a lista interna, `transferir` com destino nulo debitando antes da exceção e saldos que podiam virar `Infinity`. Todos foram corrigidos com testes.

6. Decisões iniciais

Decisão	Justificativa
Adicionar **Maven** (com *Maven Wrapper* `mvnw`) sem mover o código: `src/` continua como está; testes novos em `test/java`, cenários em `test/resources/features`.	Preserva a estrutura do professor e o projeto Eclipse; `./mvnw clean test` funciona sem instalar Maven.
JUnit 4 (4.13.2).	É a família já declarada no `.classpath` (4.12). A 4.13.2 é compatível e traz `assertThrows`.
Cucumber 7 (`cucumber-java` + `cucumber-junit`).	Integra-se ao JUnit 4 com uma única classe *runner*; permite escrever os cenários em português (`# language: pt`).
Compilar com `release 8`.	Mesma versão configurada no Eclipse. Nenhuma sintaxe posterior ao Java 8 foi usada.
Operações do `SistemaBancario` devolvem `boolean` (sucesso/rejeição).	É o padrão já usado em `ativarCliente`, `desativarCliente`, `remove` e `transfereValor`.
O diretório **não é repositório Git**.	Não foi criado repositório. A evolução do TDD está registrada em `docs/evidencias/`, um arquivo de saída real por execução, numerados em ordem cronológica. Uma cópia do código recebido está em `codigo-original/`, com o diff em `diff-original-para-final.diff`.

7. Regras assumidas (não especificadas no código nem na documentação)

Regra	Suposição adotada
Identificador	O `id` identifica o cliente; o `id` da conta é o seu número. **Não é permitido cadastrar dois com o mesmo id** (cadastro devolve `false`).
Idade	**Não foi necessário assumir**: a regra [18, 65] já existia (`Cliente.IDADE_MINIMA`/`IDADE_MAXIMA`).
Transferência de valor zero	Considerada **inválida** (rejeitada), conforme pedido no enunciado da atividade.
Transferência e situação da conta	**Não** foi criada regra exigindo conta ativa para transferir, pois ela não existe no projeto.
Remoção	**Sem restrições**, como no código original: não se exige cliente inativo, conta sem saldo etc.
Remover cliente com conta	Remover o cliente **não** remove a conta associada (comportamento original, apenas documentado por teste).
Ativar cliente já ativo / desativar já inativo	Operação idempotente: devolve `true` e mantém a situação (comportamento original).